

MCP Usage Explanation

Scope Scout (Sawyer) · The Scope Scouts · v1 · 2026-05-28

WHICH MCP SURFACES WERE USED

Scope Scout uses the Loomi Connect Marketing MCP (<https://loomi-mcp-alpha.bloomreach.com/mcp>) as its sole live workspace surface. Two other MCP surfaces were available at kickoff: the Analytics MCP (a sibling endpoint on the same domain) and the Conversation Tools MCP (a separate endpoint). Both were deliberately scoped out of v1: rather than spreading usage thin across multiple endpoints, the team elected to use the Marketing MCP deeply enough to demonstrate genuine agentic reasoning against live workspace state. For Track 2 (Engagement Intelligence and Workspace Diagnostics), the Marketing MCP is the designated primary surface; depth over breadth is the right rubric posture. Both Analytics MCP and Conversation Tools MCP are identified for roadmap consideration.

Four Marketing MCP tools are registered in the live runtime and called during every reasoning session against a live workspace:

- **get_project_overview** — Workspace identity and scale (customer count, event volume, project ID).
- **get_event_schema** — Returns the full event schema for the workspace; used as the substrate for event-family and attribute-gap detection. Per-event lookups are handled by the local `get_event_definition` tool.
- **get_mapping** — The standard taxonomy mapping table — used in mapping-ambiguity detection.
- **get_customer_property_schema** — Customer profile attribute schemas — used in attribute-availability checks.

WHY THE MARKETING MCP

The Marketing MCP exposes 81 workspace-introspection tools that answer the question: “what is true in this client’s workspace right now?” That signal is the missing input in pre-implementation scoping. Architects today either ask the client — slow, error-prone, dependent on who answers — or wait until implementation to discover gaps that should have surfaced at scoping. The Marketing MCP collapses that loop. The agent inspects the workspace before the engagement begins.

WHAT VALUE IT PROVIDES

The architectural move that makes Scope Scout an agent rather than a lookup tool is cross-reference: the agent queries local knowledge (“what should be true for this use case, per the Bloomreach Plug & Play library”) and probes the live Marketing MCP (“what is true in this workspace”) in the same reasoning step. The discovery agenda is the diff between the two.

That diff classifies into four typed gaps:

Gap Type	Description	Demo Finding
Event-family gap	The required event type does not exist in the workspace.	double_opt_in not present (blocks SDOI)
Attribute gap	The event exists but lacks a required field.	consent_list absent on consent event (blocks DOI flow)
Integration gap	Data flows exist but in the wrong direction or latency.	SMS provider not confirmed in workspace schema
Mapping ambiguity	Events flow under custom names not mapped to standard taxonomy.	All 12 standard taxonomy slots null despite 27 event types flowing

Mapping ambiguity proved to be the defining condition of the demo workspace — a finding that manual inspection missed and live MCP probing surfaced. The original manual inventory recorded 21 event types and flagged view_item as absent. The live MCP probe returned 27 event types, with view_item carrying 89,717 events. The mapping table returned event: null for all 12 standard taxonomy slots, confirming the workspace’s load-bearing condition: rich behavioral data is flowing, but none of it is wired to the standard taxonomy. That single MCP-derived finding reframed the discovery agenda for the demo client.

HOW DEEPLY THE MCP WAS USED

Scope Scout uses MCP responses as inputs to multi-step reasoning rather than as terminal outputs. A single client request triggers multiple MCP calls across an iterative agentic reasoning loop, with each call informing the next based on what the previous returned. The loop terminates when the discovery agenda is complete, or at a 25-iteration safety cap, whichever comes first. The vague-input test on the demo workspace produced more than 20 tool calls in a single session.

The agent batches multiple tool calls into a single reasoning step, visible in the console's reasoning trace. Execution is serialized and, against the live workspace, rate-paced at approximately 1.2 seconds per call to stay within the platform's 1 req/sec limit. This pacing is a deployment constraint, not an architectural one: the agent's reasoning structure is the same in fixture and live modes.

On the demo workspace: the agent called `get_project_overview`, `get_event_schema`, `get_mapping`, and `get_customer_property_schema` in a single batched reasoning step on first contact, then iteratively called `get_event_definition (local)` for specific event types, `list_use_cases` by family, `get_use_case` by code, and `find_use_cases_sharing_event` as each result opened a new line of reasoning. The session concluded only when the agent determined the discovery agenda was complete — not when a fixed number of passes had run.

OPERATIONAL CONSIDERATIONS

Rate limit

The Loomi Connect Marketing MCP enforces 1 request per second. The runtime serializes live MCP calls with 1.2-second pacing to stay reliably under the cap. Fixture mode (used for the demo recording) is not rate-constrained.

Authentication

OAuth via the mcp-remote bridge. No credentials are stored in the codebase or logged at any tier. The runtime explicitly suppresses logging of the workspace introspection response to prevent access token exposure.

Read-only posture

All MCP calls are read-only by design. Every registered tool maps to a `get_*` or `list_*` read operation. Scope Scout never writes to the workspace. The system prompt reinforces this posture explicitly.

Access logging

Every MCP tool invocation emits a structured access line to server-side stderr: timestamp, tool name, source (local or live-mcp), query parameters, response size in bytes, status (ok/error), and latency in milliseconds. Response payloads are never written to the access log. Verbose response logging is available behind a development flag (`SCOPE_SCOUT_VERBOSE_LOG=1`) and is disabled in all demo and production configurations.

WHAT THIS ENABLES

For the architect, the MCP integration turns scoping from a deductive exercise (“what should this workspace need?”) into an evidentiary one (“what does this workspace have, what does it lack, and where are the load-bearing gaps?”). The discovery agenda Scope Scout produces is grounded in workspace evidence rather than in client assertion. That is the difference between an architect-quality scope and a client-asserted scope, and it is the difference the MCP integration makes possible.